Московский Авиационный Институт

(Национальный Исследовательский Университет)

Институт №8 “Компьютерные науки и прикладная математика”

Кафедра №806 “Вычислительная математика и программирование”

Лабораторная работа №X по курсу

«Операционные системы»

Группа: М8О-212БВ-25

Студент: Морозов Д. А.

Преподаватель: Бахарев В.Д.

Оценка: _____

Дата: 22.10.26

Москва, 2026

Постановка задачи

Вариант 12.

Родительский процесс создает два дочерних процесса. Родительский процесс принимает от пользователя строки произвольной длины и пересылает их в pipe1. Процесс child1 переводит строки в верхний регистр. Процесс child2 убирает все задвоенные пробелы. Child2 пересылает результат своей работы родительскому процессу через pipe2. Родительский процесс полученный результат выводит в стандартный поток вывода. Родительский и дочерний процессы представлены разными программами.

Общий метод и алгоритм решения

Использованные системные вызовы:

- `fork()` — создает дочерние процессы.
- `pipe()` — создает каналы (трубы) для передачи данных между процессами.
- `dup2()` — перенаправляет стандартные потоки ввода/вывода (stdin, stdout) на концы каналов.
- `execlp()` — запускает новые программы (`child1` и `child2`), заменяя ими текущий процесс.
- `read()` / `write()` — чтение и запись данных в каналы и на экран.
- `close()` — закрывает неиспользуемые дескрипторы (чтобы процессы получали сигнал конца данных EOF).
- `waitpid()` — ожидает завершения дочерних процессов.

Алгоритм работы:
Программа разделена на три отдельных исполняемых файла, работающих в конвейере.

1. `parent.c` создает три канала и с помощью `fork()` порождает двух детей. Через `execlp()` запускаются программы `child1` и `child2`, у которых с помощью `dup2()` stdin и stdout перенаправлены на каналы. Родитель читает строки от пользователя и передает их первому ребенку, после чего читает готовый результат от второго и выводит на экран.
2. `child1.c` читает данные из своего stdin (канал от родителя), переводит все строчные буквы в заглавные и пишет результат в stdout (канал ко второму ребенку).
3. `child2.c` читает данные от `child1`, удаляет повторяющиеся пробелы (оставляя по одному) и передает очищенный текст родителю через третий канал.

Код программы

child1.c

```
#include <unistd.h>
#include <stdlib.h>


int main(void) {
    char buffer[4096];
    ssize_t bytes;
    while (1) {
        bytes = read(STDIN_FILENO, buffer, sizeof(buffer));
        if (bytes == -1) {
            write(2, "Ошибка чтения в child1\n", 24);
            exit(-1);
        } else if (bytes == 0) {
            break;
        }
        for (ssize_t i = 0; i < bytes; ++i) {
            if (buffer[i] >= 'a' && buffer[i] <= 'z') {
                buffer[i] -= 32;
            }
        }

        if (write(STDOUT_FILENO, buffer, bytes) == -1) {
            write(2, "Ошибка записи в child1\n", 24);
            exit(-1);
        }
    }

    return 0;
}
```

child2.c

```
#include <unistd.h>
#include <stdlib.h>

int main(void) {
    char buffer[4096];
    char out[4096];
    ssize_t bytes;
    int prev_space = 0;

    while (1) {
        bytes = read(STDIN_FILENO, buffer, sizeof(buffer));
        if (bytes == -1) {
            write(2, "Ошибка чтения в child2\n", 24);
            exit(-1);
        } else if (bytes == 0) {
            break;
        }

        ssize_t k = 0;
        for (ssize_t i = 0; i < bytes; ++i) {
            if (buffer[i] == ' ') {
                if (!prev_space) {
                    out[k++] = ' ';
                    prev_space = 1;
                }
            } else {
                out[k++] = buffer[i];
                prev_space = 0;
            }
```

```
        }

        if (k > 0 && write(STDOUT_FILENO, out, k) == -1) {
            write(2, "Ошибка записи в child2\n", 24);
            exit(-1);
        }
    }

    return 0;
}
```

parent.c

```
#include <fcntl.h>
#include <stdlib.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void) {
    int pipe1[2];
    int pipe2[2];
    int pipe3[2];

    if (pipe(pipe1) == -1 || pipe(pipe2) == -1 || pipe(pipe3) == -1) {
        write(2, "Ошибка создания pipe\n", 20);
        exit(-1);
    }

    pid_t child1 = fork();
    if (child1 == -1) {
        write(2, "Ошибка fork (child1)\n", 21);
        exit(-1);
```

```
} else if (child1 == 0) {
    if (dup2(pipe1[0], STDIN_FILENO) == -1 ||
        dup2(pipe2[1], STDOUT_FILENO) == -1) {
        write(2, "Ошибка dup2 в child1\n", 21);
        exit(-1);
    }
    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    close(pipe3[0]); close(pipe3[1]);

    execlp("./child1", "child1", NULL);
    write(2, "Ошибка запуска child1\n", 22);
    exit(-1);
}

pid_t child2 = fork();
if (child2 == -1) {
    write(2, "Ошибка fork (child2)\n", 21);
    exit(-1);
} else if (child2 == 0) {
    if (dup2(pipe2[0], STDIN_FILENO) == -1 ||
        dup2(pipe3[1], STDOUT_FILENO) == -1) {
        write(2, "Ошибка dup2 в child2\n", 21);
        exit(-1);
    }
    close(pipe1[0]); close(pipe1[1]);
    close(pipe2[0]); close(pipe2[1]);
    close(pipe3[0]); close(pipe3[1]);

    execlp("./child2", "child2", NULL);
```

```
    write(2, "Ошибка запуска child2\n", 22);
    exit(-1);
}

close(pipe1[0]);
close(pipe2[0]);
close(pipe2[1]);
close(pipe3[1]);

char buffer[2048];
ssize_t bytes;
while (1) {
    bytes = read(STDIN_FILENO, buffer, sizeof(buffer));
    if (bytes == -1) {
        write(2, "Ошибка чтения из stdin\n", 23);
        exit(-1);
    }
    if (bytes == 0) {
        break;
    }
    if (write(pipe1[1], buffer, bytes) == -1) {
        write(2, "Ошибка записи в pipe1\n", 22);
        exit(-1);
    }
}
close(pipe1[1]);

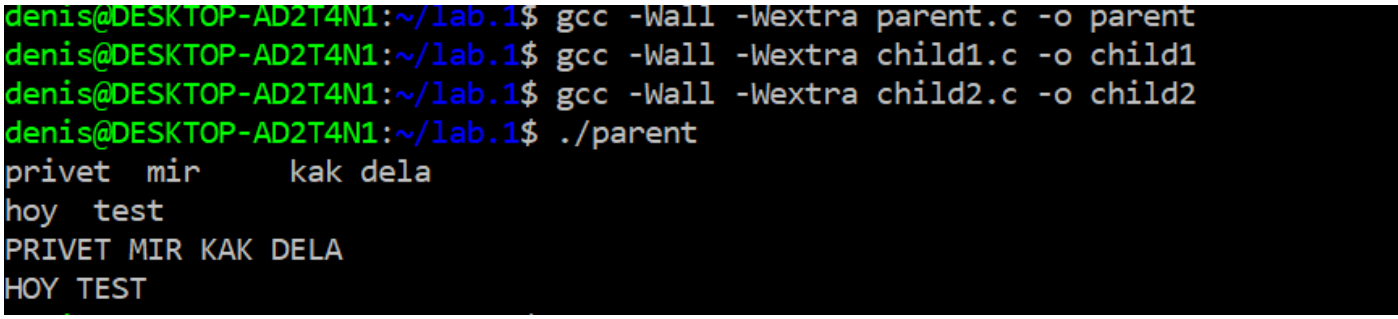
while (1) {
    bytes = read(pipe3[0], buffer, sizeof(buffer));
    if (bytes == -1) {
```

```
            write(2, "Ошибка чтения из pipe3\n", 23);
            exit(-1);
        }
        if (bytes == 0) {
            break;
        }
        if (write(STDOUT_FILENO, buffer, bytes) == -1) {
            write(2, "Ошибка записи в stdout\n", 23);
            exit(-1);
        }
    }
    close(pipe3[0]);

    waitpid(child1, NULL, 0);
    waitpid(child2, NULL, 0);

    return 0;
}
```

Протокол работы программы



Это подтверждает, что первый дочерний процесс корректно перевел все буквы в верхний регистр, а второй дочерний процесс успешно удалил лишние пробелы. Взаимодействие через каналы работает корректно.

Вывод

В ходе выполнения лабораторной работы были изучены механизмы межпроцессного взаимодействия в ОС Linux. Были получены практические навыки создания дочерних процессов с помощью системного вызова `fork()` и замены исполняемого файла с помощью `execlp()`. Также было освоено создание неименованных каналов (`pipe`) и перенаправление стандартных потоков ввода-вывода (`dup2`).

Основной сложностью при выполнении работы было корректное закрытие файловых дескрипторов во всех процессах, так как незакрытые концы каналов приводили к зависанию программы в ожидании данных. Тем не менее, задача была успешно решена, программа корректно обрабатывает строки произвольной длины и завершается по сигналу конца файла.